

Assignment # 2 – Designing Artificial Neural Networks and Convolutional Neural Network for CIFAR-10

Instructor: Dr. Ahmad Raza Shahid

Course Information

Course: Deep Learning for Perception (CS4045)

Assignment: Experimental ANN and CNN design for CIFAR-10 classification

Due Date: As announced on Google Classroom (GCR)

Total Points: 100

Format: Jupyter/Colab Notebooks + Formal Report (PDF)

Assignment Philosophy

This assignment is experimental and research-focused. The goal is to explore how far carefully designed fully-connected (non-transformer) neural networks can go on an image task (CIFAR-10).

You will:

1. Design and implement your own ANN architectures (no off-the-shelf CNNs, ViTs, pretrained models, or transfer learning allowed).
 2. Run systematic experiments (architecture choices, activation functions, normalization, regularization, optimizer settings, data preprocessing/augmentation, learning rate schedules, etc.).
 3. Report results, analysis and lessons learned in a short research-style report.
 4. Think like an experimental researcher: iterate, measure, and justify every choice.
-

Dataset

CIFAR-10: 60,000 color images (32×32), 10 classes. Use the standard train/test split provided by the dataset. Students must evaluate final model accuracy on the official test set (10,000 images).

Rules & Constraints (strict)

- Allowed model family: Fully-connected feedforward ANN architectures only (dense layers). You may use clever architectural ideas (wide/deep stacks, skip connections implemented via linear layers, gating, feature-wise transforms, learned positional

encodings, residual MLP blocks, etc.) and for CNN you can use convolutional layer, pooling layer, batch normalization layer and dropout layer but attention layers, Vision Transformers, or any model architecture that is explicitly transformer-based.

- No pretrained weights or transfer learning from other models or datasets.
 - No use of standard CNN architectures (AlexNet, VGG, ResNet, EfficientNet, MobileNet, etc.) or libraries that implicitly call them for you.
 - **Data augmentation is allowed and encouraged** (random crops, flips, color jittering, mixup, cutout, etc.). Describe augmentation in your notebook and report.
 - You may use common building blocks from frameworks (PyTorch, TensorFlow/Keras) for implementation, training loops, optimizers, schedulers, and data loading.
 - **Reproducibility:** set and report random seeds; ensure code runs from top to bottom and produces the reported numbers.
-

TASK

In addition to the fully-connected ANN design:

1. **Implement CIFAR-10 using both ANN and CNN architectures and compare their performance.**
 - Analyze which model classifies better and explain why (in terms of representational capacity, inductive bias, and feature extraction).
 - Achieving $\geq 70\%$ accuracy on the test set earns full marks.
 2. **Implement a Hybrid CNN + ANN model, where the CNN acts as a feature extractor and the ANN serves as a classifier.**
 - Evaluate how efficiently this hybrid model performs compared to standalone ANN and CNN.
 - Discuss trade-offs in complexity, performance, and training stability.
-

Objective and Grading Policy

Objective: Achieve as high a top-1 accuracy on CIFAR-10 test set as possible using your ANN and CNN designs.

Grading (100 points total):

- Accuracy $\geq 70\%$ $\rightarrow$ 100 points (full marks)
- $60\% \leq \text{Accuracy} < 70\%$ $\rightarrow$ linear scaling between 70–95 points
- $50\% \leq \text{Accuracy} < 60\%$ $\rightarrow$ linear scaling between 35–69 points
- $45\% \leq \text{Accuracy} < 50\%$ $\rightarrow$ 50% of above bucket points

- Accuracy < 45% → 0 points
-

Deliverables

You must submit:

1. **Notebooks (Primary deliverable):** one or more runnable Jupyter/Colab notebooks that:
 - Implement all three models (ANN, CNN, Hybrid CNN+ANN).
 - Include code cells that reproduce the reported final evaluation (provide a checkpoint or training script).
 - Clearly annotate architecture design and hyperparameters.
 2. **Formal Report (PDF, 4–8 pages):** should contain:
 - Abstract (1 paragraph)
 - Model architecture description (diagrams or layer tables)
 - Training recipe (optimizers, LR schedule, batch size, epochs, augmentations)
 - Ablation / experiments (what you tried and why)
 - Final results (test accuracy, confusion matrix) and analysis
 - Discussion: what worked, what didn't, limitations, and possible improvements
 3. **Model checkpoint(s):** include final trained weights (or link).
 4. **README.txt:** short instructions on how to run the notebooks and reproduce final results.
-

Suggested Experimental Directions (Non-Exhaustive)

These are suggestions — originality is rewarded:

- Input preprocessing: flattening pixels vs learned linear embeddings, per-channel normalization, learned projections.
- Large width MLPs vs deep narrow MLPs. Residual MLP blocks.
- Activation functions: ReLU, Leaky ReLU, GELU, Swish, SELU.
- Normalization: BatchNorm on linear activations, LayerNorm, or feature-wise transforms.
- Regularization: Dropout, weight decay, label smoothing, mixup, cutout.
- Optimizers: SGD with momentum, Adam, AdamW; learning-rate schedules (cosine, step decay, warmup).
- Advanced tricks: progressive resizing, ensemble of ANNs, gradient clipping, curriculum learning.

Document every experiment and hyperparameter change.

Evaluation Procedure and Reproducibility

- Instructors will run the student-provided evaluation script (or notebook cell) to verify the reported test accuracy. Include instructions and a checkpoint.
 - If the submitted code does not reproduce the reported accuracy, grading will use the verified accuracy obtained by the instructors.
 - Make sure random seeds are set (numpy, torch/tf, python) and reported in README.
-

Academic Integrity and Collaboration

- Collaboration on ideas is allowed, but the submitted code and report must be your own work.
 - Copying architectures or training recipes from published works, blogs, or classmates without attribution is not allowed.
 - Any detected plagiarism will receive zero marks and be reported to the department.
-

Tips for Success

- Start with a simple MLP baseline and strengthen it incrementally.
 - Keep experiments organized (table of runs, hyperparameters, and results).
 - Use smaller pilot runs to find reasonable hyperparameters before full training.
 - Focus on stability and reproducibility: document seeds, environment, and random behavior.
-

Deadline and Submission

Due Date: As announced on Google Classroom (GCR).

Submission: Submit a single ZIP file named

i24xxxx_name_A2.zip

containing:

- notebooks/
 - report.pdf
 - checkpoints/
 - README.txt
- via the course submission portal.